Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Proving Basic Algorithms Correct: Sorting
Computer-Assisted Theorem Proving (7CCSACTP)
Mohammad Abdulaziz

Computation Induction

So far: *structural induction*, since all functions were primitive recursive.
But `merge` shortens `xs` in one recursive call and `ys` in the other.

Computation Induction
So far: structural induction, since all functions were primitive recursive.
But merge shortens xs in one recursive call and ys in the other.

- Based on Nipkow's Isabelle/HOL slides (Computation Induction, How to apply f.induct, Isar naming).
- So far every function was primitive recursive: each recursive call removes one constructor, each induction step adds one. merge is not: its second recursive call keeps x # xs and shortens ys instead, so structural induction on xs alone does not match it.
- The induction follows the course of the (terminating!) computation. Motto: properties of f are best proved by rule f.induct.
- Structural induction on lists has one case per constructor: `P []`, and `P xs` $\Longrightarrow$ `P (x # xs)`. Computation induction instead has one case per defining equation of the function, with one IH per recursive call.
- **Definition:** if `f` is defined by **fun**, Isabelle provides the rule `f.induct`: for each defining equation `f e = ... f` r_1 `...  f` r_k `...`, prove `P e` assuming `P` r_1, ..., `P` r_k.

Computation Induction: Example

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
```

Prove `length (merge xs ys) = length xs + length ys`
by `merge.induct`, with `P xs ys` this property: show

1. `P [] ys`
2. `P (x # xs) []`
3. `(x ≤ y ⟹ P xs (y # ys)) ⟹`
   `(¬ x ≤ y ⟹ P (x # xs) ys) ⟹`
   `P (x # xs) (y # ys)`

In Isabelle: `proof (induction xs ys rule: merge.induct)`
Cases are named `1`, `2`, `3`; the IH of case 3 is `"3.IH"` (with quotes).

Computation Induction: Example

merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
(if x ≤ y then x # merge xs (y # ys)
else y # merge (x # xs) ys)

Prove length (merge xs ys) = length xs + length ys
by merge.induct, with P xs ys this property: show

1. P [] ys
2. P (x # xs) []
3. (x ≤ y ⟹ P xs (y # ys)) ⟹
(¬ x ≤ y ⟹ P (x # xs) ys) ⟹
P (x # xs) (y # ys)

In Isabelle: proof (induction xs ys rule: merge.induct)
Cases are named 1, 2, 3; the IH of case 3 is "3.IH" (with quotes).

- One case per defining equation of merge, with one IH per recursive call.
- Case 2 is `x#xs` and `[]`: the first two equations overlap on `merge [] []`, and Isabelle makes them disjoint.
- The IH of case 3 has the conditions of the **if**: each recursive call is only made in its branch.
- Heuristic: the goal should contain a call f a1 . . . an, ideally with variables as the ai.
- Case 3, branch x ≤ y: length (x # merge xs (y # ys)) = 1 + length xs + length (y # ys) by the first IH; the other branch is symmetric.

Informal Proof: Length of `merge`

Lemma

```
length (merge xs ys) = length xs + length ys
```


Informal Proof: Length of merge
Lemma
length (merge xs ys) = length xs + length ys

Proof by computation induction (rule `merge.induct`):

- *Case 1:* $|\text{merge } [] \ ys| = |ys| = |[]| + |ys|$.
- *Case 2:* $|\text{merge } (x\#xs) \ []| = |x\#xs| = |x\#xs| + |[]|$.
- *Case 3,* $x \leq y$*:*

$$\begin{aligned} |\text{merge } (x\#xs) \ (y\#ys)| &= |x\#\text{merge } xs \ (y\#ys)| = 1 + |\text{merge } xs \ (y\#ys)| \\ &= 1 + |xs| + |y\#ys| \quad \text{[IH for } x \leq y\text{]} \\ &= |x\#xs| + |y\#ys| \end{aligned}$$

- *Case 3,* $\neg\, x \leq y$*:* symmetric, with the IH for $\neg\, x \leq y$:
  $|y\#\text{merge } (x\#xs) \ ys| = 1 + |x\#xs| + |ys| = |x\#xs| + |y\#ys|$. □
- In Isabelle: `by (induction xs ys rule: merge.induct) auto`.